

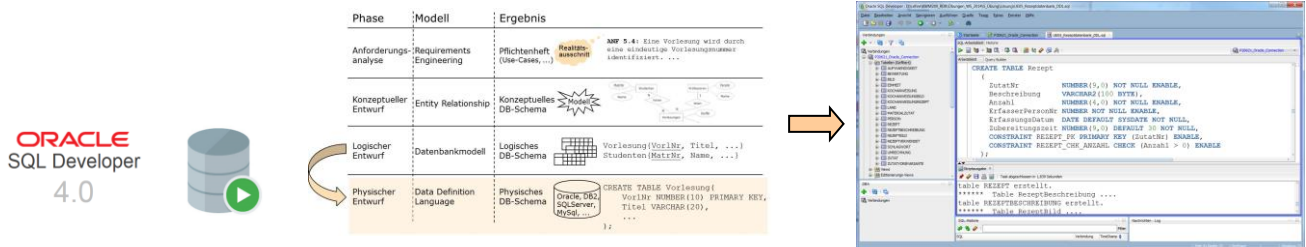
Abgabetermine: 30.4.2026, 12 Uhr
Abgabe elektronisch im e-learning

☒ DEM2G1 Dr. Pitzer Name Klemens Danner Aufwand in h 11
☐ DEM2G2 Dr. Pitzer
☐ DEM2G3 Dr. Niklas Punkte _____ Kurzzeichen Tutor _____

Hinweise und Richtlinien:

- Übungsausarbeitungen müssen den im Syllabus angegebenen Formatierungsrichtlinien entsprechen – Nichtbeachtung dieser Formatierungsrichtlinien führt zu Punkteabzug.
- Zusätzlich zu den allgemeinen Formatierungsrichtlinien sind für diese Übungsausarbeitung folgende zusätzlichen Richtlinien zu beachten:
 - Verwenden Sie für die Erstellung des physischen Entwurfs das **Werkzeug SQL Developer** und die **Datenbank Oracle**!
 - Die zu erstellenden SQL-Skripte sind gemäß den Formatierungsrichtlinien als Textdatei hochzuladen.
 - Treffen Sie, falls notwendig, sinnvolle Annahmen und dokumentieren Sie diese nachvollziehbar in ihrer Lösung!

Ziel dieser Übung ist es, das logische Schema (= relationale Modell) in ein physisches Schema zu überführen. Das physische Schema soll für das Datenbanksystem Oracle 19c erstellt werden.



Rezeptdatenbank

Gesunde Ernährung ist ein wichtiger Faktor, um auch gute Leistungen beim Studium zu erbringen. In diesem Sinne möchten wir eine Rezeptdatenbank mit interessanten, wohlschmeckenden und leicht zu kochenden Studentenrezepten zusammenstellen.

Folgende Aufgabenstellung bzw. Anforderungen sollen durch eine Rezeptdatenbank abgedeckt werden:

- Die zentrale Komponente unserer Informationssammlung stellt ein Rezept dar. Für ein Rezept sind der Name [„Lasagne“], eine Beschreibung [„ein Klassiker“] und die eintragende Person zu speichern. Ein Rezept wird immer für eine bestimmte Personenanzahl in der Basisvariante erstellt [z.B. Lasagne für vier Personen].
- Beachten Sie bei der Modellierung, dass es natürlich vorkommen kann, dass die geplante Party und die dazugehörige Anzahl an Lasagne-Portionen schnell von 10 auf 30 explodieren können. Die Datenbank muss eine einfache Möglichkeit bieten, die Rezepte ausgehend von der Anzahl der Basispersonen umzurechnen. (Hierbei muss natürlich kein Algorithmus implementiert werden, sondern es müssen nur alle Daten vorhanden sein, um so eine Berechnung möglichst einfach durchführen zu können.)
- Ein Rezept verwendet Zutaten in bestimmten Mengenangaben. Eine Zutat ist durch den Namen [„Zwiebel“] und eine eventuelle Vorbereitungsvariante [„klein gehackt“] definiert. Eine Zutat hat immer einen Bezug auf eine Grundeinheit [„Stk“]. Bei der Verwendung im Rezept wird die wirkliche Menge angegeben [„2“]. Eine Zutat kann aber wiederum selbst ein Rezept sein [z.B. „Sauce Béchamel“, welche in einem Rezept [unserer Lasagne] verwendet wird [1 Portion Sauce für das Lasagne-Rezept]. Zutaten, die kein Rezept sind, können einen Einkaufspreis aufweisen.
- Personen können für Rezepte Labels vergeben und diese somit genauer beschreiben [z.B. Partyrezept, große Mengen möglich, romantisches Dinner, vegan, vegetarisch, scharf, usw.].
- Einheiten können auch in andere Einheiten umgerechnet werden [z.B. ml nach Liter, Gramm nach Kilo]. Zu diesem Zweck sollte für jede Einheit ein Umrechnungsfaktor in andere Einheiten abgespeichert werden können. Hinweis: es müssen nur einfache Umrechnungen möglich sein, z.B. ist es nicht notwendig z.B. von Cups auf Gramm umzurechnen, da man dazu die Dichte der Substanz kennen müsste.

Beispiel: *Liter in ml → Faktor 1000*
 ml in Liter → Faktor 0,001

- Ein Rezept besteht aus einer Folge von Anweisungen, wie die Speise zubereitet werden sollte. Diese Anweisungen enthalten eine textuelle Beschreibung des Schrittes und sind in einer bestimmten Reihenfolge [1. Zwiebeln anrösten, 2. Fleisch anrösten, usw.] durchzuführen. Zu jeder Anweisung ist die benötigte Zubereitungszeit vermerkt.
- *Ein Bild sagt mehr als tausend Worte!* Auch für unsere Rezeptdatenbank gilt: wir wollen unsere Schritte möglichst genau dokumentieren können, um auch weniger begabten Hobbyköchen eine Möglichkeit zum Nachkochen zu bieten. Bilder sind durch einen Dateinamen und einem Verzeichnis, in dem das Bild gespeichert ist, definiert. Rezeptanweisungen aber auch die Rezepte selbst können durch Bilder genauer beschrieben werden. Mehrere Bilder pro Anweisung / Rezept sind möglich – eine Reihenfolge der Bilder sollte definierbar sein.
- Die Qualität der Rezepte möchten wir durch ein Bewertungssystem testen. Für Rezepte können Bewertungen im Schulnotensystem (1-5) für die Kategorien *Geschmack* und *Einfachheit* vergeben werden. Der Zeitpunkt der Bewertung und die bewertende Person sind ebenfalls abzuspeichern. Ein optionaler Kommentar rundet die Bewertung ab.

Rezeptdatenbank – Konzeptuelles Modell

Ein erster Modellierungs-Workshop hat folgendes konzeptuelles Modell ergeben.

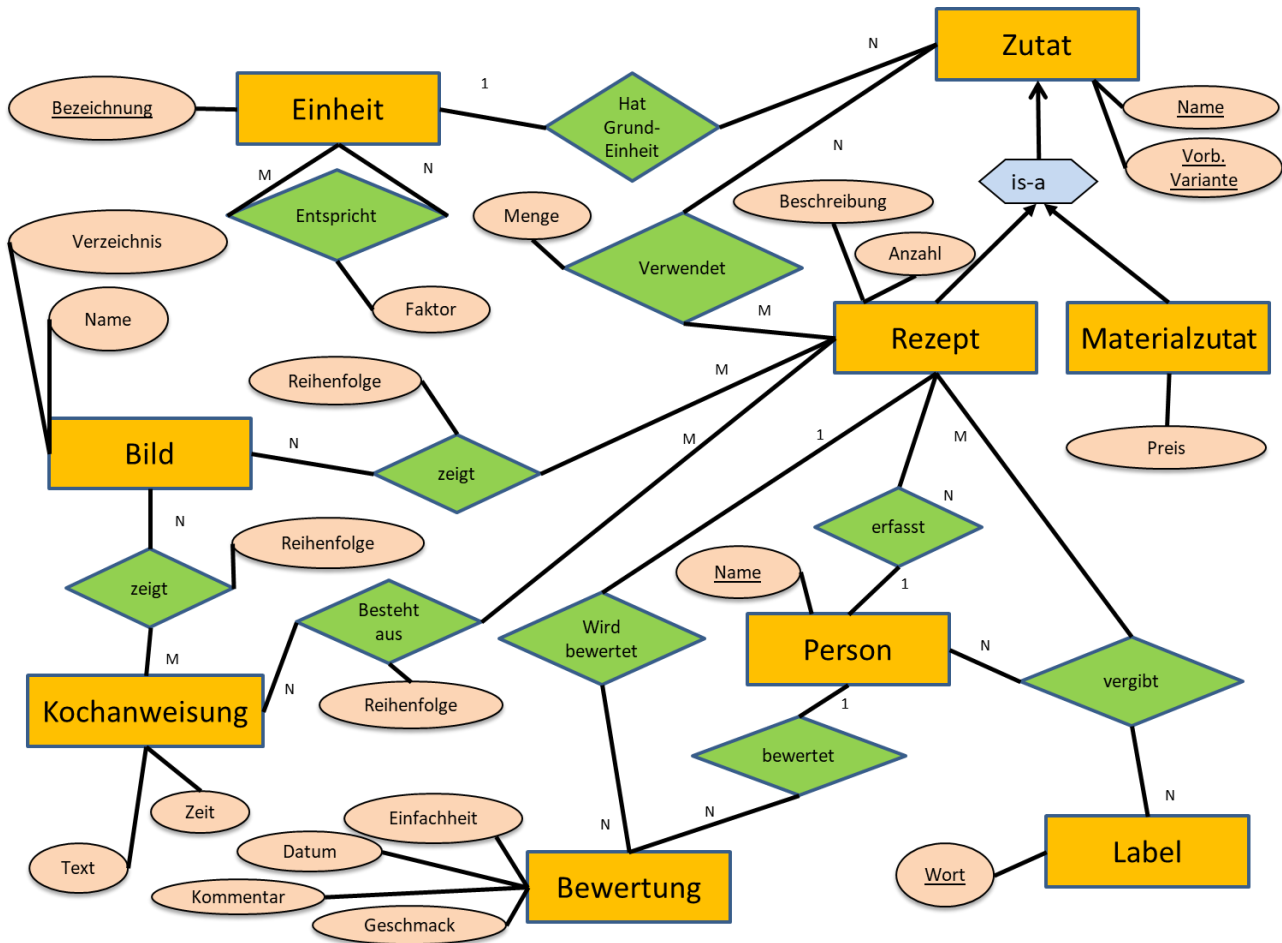


Abbildung 1: Rezeptdatenbank – Konzeptuelles Modell

1. Rezeptdatenbank – Relationales Modell

(7 Punkte)

Überführen Sie das Entity Relationship Diagramm in ein relationales Modell. Ergänzen Sie das Modell um fehlende Attribute bzw. Schlüssel und kennzeichnen Sie Primär- und Fremdschlüssel (und auch die Kombination der beiden). Führen Sie künstliche Schlüssel ein, wo Sie es für sinnvoll erachten. Das relationale Modell muss der 3. Normalform genügen. Die Darstellung des relationalen Modells hat in der folgenden Notation zu erfolgen:

```
Relation1(Attribut1:Datentyp1, Attribut2:Datentyp2, ...)
```

```
Relation2(Attribut3:Datentyp3, NeuerFkName:Datentyp4 -> OriginalFkName.Relation)
```

Wichtig: Bei dieser Aufgabe sind zusätzlich die Datentypen mit anzugeben! Wählen Sie sinnvolle Datentypen für die Attribute aus! Treffen Sie, falls notwendig, sinnvolle Annahmen und dokumentieren Sie diese nachvollziehbar in ihrer Lösung!

2. Rezeptdatenbank – Physisches Modell

(8 Punkte)

Erstellen Sie auf Basis des relationalen Modells aus Aufgabe 1b) das physische Datenbankschema für das Datenbanksystem Oracle 19c.

- Verwenden Sie zur Erstellung des Datenbankschemas die Datendefinitionssprache (DDL) von SQL und speichern Sie die Anweisungen in einem SQL-Skript.
- Erzeugen Sie mit diesem Skript das Datenbankschema in der Oracle-Datenbank.
- Für diese Aufgabe ist die Definition von **Fremdschlüsselbeziehungen** zwischen den Tabellen vorerst **zu vernachlässigen!**
- Beachten Sie bei der Definition der DDL u.a. folgende Punkte:
 - Überlegen Sie, welche Attribute keine NULL-Werte annehmen dürfen. Vergeben Sie Default-Werte, wo Sie aus ihrer Sicht sinnvoll sind.
 - Falls notwendig, definieren Sie attributspezifische Bedingungen (bspw. CHECK-Klauseln für das Schulnotensystem ≥ 1 und ≤ 5).
 - Überlegen Sie, für welche Attribute / Attributkombinationen jeder Wert nur einmal vorkommen darf.
 - Definieren Sie Attribute und Attributkombinationen als Primärschlüssel.
 - Vergeben Sie sinnvolle Namen für alle Constraints.

Speichern Sie die Anweisungen für das Erstellen des Datenbankschemas in der SQL-Datei `CREATE_Rezeptdatenbank.sql`.

3. Rezeptdatenbank – Schemaevolution

(4 Punkte)

Das in Aufgabe 2 erstellte physische Datenbankschema ist um die Fremdschlüsselbeziehungen zu erweitern.

- Erstellen Sie ein SQL-Skript, in welchem die fehlenden Fremdschlüsselbeziehungen mit `ALTER TABLE`-Anweisungen erstellt werden.
- Achten Sie darauf, dass alle notwendigen (d.h. lt. relationalem Modell geforderten) Fremdschlüsselbeziehungen angelegt werden.
- Die Möglichkeit zu einer Bewertung eines Geschmacks zu erfassen soll entfernt werden. Verwenden Sie dazu eine `ALTER TABLE`-Anweisung.
- Führen Sie eine zusätzliche Bedingung ein, dass ein Rezept nur für maximal 12 Personen definiert werden darf.
- Testen Sie die Anweisungen und überprüfen Sie, ob die Änderungen erfolgreich durchgeführt wurden (z.B. durch die Abfrage des Data-Dictionary im SQL Developer).

Speichern Sie die Anweisungen für das Erweitern um Fremdschlüsselbeziehungen in der SQL-Datei `ALTER_Rezeptdatenbank.sql`.

4. Rezeptdatenbank – Löschen von Tabellen

(1 Punkt)

Die in der Aufgabe 2 erzeugten bzw. in Aufgabe 3 erweiterten Tabellen sind mit einem SQL-Skript zu löschen.

- **Vorbedingung:** die Fremdschlüsselbeziehungen aus Aufgabe 3 sind in der Datenbank enthalten.
- Erstellen Sie ein SQL-Skript, welches die Tabellen in der Datenbank löscht!
- Kontrollieren Sie die erfolgreiche Löschung durch die Abfrage des Data-Dictionary mit Hilfe des Werkzeuges SQL Developer.
- Das Skript muss mit **einem** Durchlauf alle Tabellen löschen! Ist das Lösch-Skript fehlerhaft (d.h. es werden nicht alle Tabellen gelöscht; das Skript wurde nicht getestet usw.) werden keine Punkte für diese Aufgabe vergeben!

Speichern Sie die Anweisungen für das Löschen des gesamten Datenbankschemas in der SQL-Datei `DROP_Rezeptdatenbank.sql`.

5. Erstellung DDL für gegebene Daten

(4 Punkte)

Die CSV-Datei `Messungen.csv` enthält Messwerte von Hardware-Sensoren und zugehörige Geräteinformationen. Analysieren Sie die Daten und überführen Sie diese in ein **relationales Modell**, das den Anforderungen der **dritten Normalform** entspricht. Erstellen Sie anschließend die notwendigen **DDL-Statements** zur Anlage der Datenbankobjekte und berücksichtigen Sie dabei folgende Anforderungen:

- Wählen Sie passende Datentypen für alle Attribute
- Definieren Sie Primärschlüssel und Fremdschlüssel.
- Beim Ausbau eines Geräts (Löschen), sollen auch alle zugehörigen Messwerte entfernt werden.
- Die laufenden Kosten eines Geräts dürfen nicht kleiner als 0 sein.
- Ein Gerät ist standardmäßig **aktiv**, sofern kein anderer Wert angegeben ist.
- Der Messwert darf nie leer sein, außer wenn sich der Status im Zustand **Fehler** befindet.

LogId	GeraeteId	GeraeteBezeichnung	EinbauDatum	Messwert	Einheit	Messdatum	Status	Aktiv	LaufendeKosten	UntererGrenzwert	ObererGrenzwert
1	101	AI-X-Alpha	2025-01-01	42.5	C-Celsius	2025-05-01 10:00:00	0-Normal	True	150.00	10.0	60.0
2	101	AI-X-Alpha	2025-01-01	65.2	C-Celsius	2025-05-01 11:00:00	3-Kritisch	True	150.00	10.0	60.0
3	102	BATT-Delta	2024-11-15	12.8	V-Volt	2025-05-02 08:30:00	0-Normal	True	89.50	11.0	14.5
4	102	BATT-Delta	2024-11-15		V-Volt	2025-05-02 09:00:00	2-Fehler	False	89.50	11.0	14.5
5	103	AI-X-Beta	2026-02-20	38.7	C-Celsius	2026-04-02 12:45:00	0-Normal	True	210.00	15.0	55.0
6	104	BATT-Zeta	2026-04-10	9.5	V-Volt	2026-04-18 14:20:00	1-Warnung	True	50.00	11.0	14.5

Abbildung 2: Datei Messungen.csv

1. Rezeptdatenbank - Relationales Modell

Zutat(ZutatID:Integer, Name:VARCHAR, VorbVariante:VARCHAR, ZEinheitID:Integer -> EinheitID.Einheit)
 Rezept(RezeptID:Integer -> ZutatID.Zutat, Beschreibung:VARCHAR, Anzahl:Integer, VerfasserID:Integer -> PersonID.Person)
 Materialzutat(MaterialID:Integer -> ZutatID.Zutat, Preis:Decimal)
 Person(PersonID:Integer, Vorname:VARCHAR, Nachname:VARCHAR) //PersonID erforderlich (zwei gleiche Namen?)
 Label(LabelID:Integer, Wort:VARCHAR)
 Bewertung(BewertungsID:Integer, BRezeptID:Integer -> RezeptID.Rezept, Einfachheit:Integer, Geschmack:Integer, Datum:Date, Kommentar:VARCHAR, BewerterID:Integer -> PersonID.Person)
 Einheit(EinheitID:Integer, Bezeichnung:VARCHAR)
 Bild(BildID:Integer, Name:VARCHAR, Verzeichnis:VARCHAR)
 Kochanweisung(AnweisungsID:Integer, Zeit:Integer, Text:VARCHAR)

vergibt(RezeptID:Integer -> RezeptID.Rezept, PersonID:Integer -> PersonID.Person, LabelID:Integer -> LabelID.Label)
 verwendet(ZutatID:Integer -> ZutatID.Zutat, RezeptID:Integer -> RezeptID.Rezept, Menge:Integer)
 Entspricht(EinheitID1:Integer -> EinheitID.Einheit, EinheitID2:Integer -> EinheitID.Einheit, Faktor:FLOAT)
 Rezept_zeigt(RezeptID:Integer -> RezeptID.Rezept, BildID:Integer -> BildID.Bild, Reihenfolge:Integer)
 bestehtAus(RezeptID:Integer -> RezeptID.Rezept, AnweisungsID:Integer -> AnweisungsID.Kochanweisung, Reihenfolge:Integer)
 Anweisung_zeigt(AnweisungsID:Integer -> AnweisungsID.Kochanweisung, BildID:Integer -> BildID.Bild, Reihenfolge:Integer)

1. NF ist erfüllt
2. NF ist auch erfüllt (alle Nichtschlüsselattribute komplett vom Schlüssel abhängig)
3. NF ist auch erfüllt (keine transitiven Abhängigkeiten)

2. Rezeptdatenbank – Physisches Modell

```

1  CREATE TABLE Zutat (
2      ZutatID NUMBER(9), --primary key
3      Name VARCHAR(50) NOT NULL,
4      VorbVariante VARCHAR(50),
5      ZEinheitID NUMBER(9) NOT NULL,
6      CONSTRAINT Zutat_PK PRIMARY KEY (ZutatID)
7  );
8
9  CREATE TABLE Rezept (
10     RezeptID NUMBER(9), --primary and foreign key
11     Beschreibung VARCHAR(1000) NOT NULL,
12     Anzahl NUMBER(3) DEFAULT 4 NOT NULL,
13     VerfasserID NUMBER(9) NOT NULL,
14     CONSTRAINT Rezept_PK PRIMARY KEY (RezeptID)
15 );
16
17 CREATE TABLE Materialzutat (
18     MaterialID NUMBER(9),
19     Preis NUMBER(8,2) NOT NULL,
20     CONSTRAINT Material_PK PRIMARY KEY (MaterialID)
21 );
22
23 CREATE TABLE Person (
24     PersonID NUMBER(9),
25     vorname VARCHAR(30),
26     nachname VARCHAR(30),
27     CONSTRAINT Person_PK PRIMARY KEY (PersonID)
28 );
29
30 CREATE TABLE Label (
31     LabelID NUMBER(9),
32     Wort VARCHAR(30) NOT NULL,
33     CONSTRAINT Label_PK PRIMARY KEY (LabelID)
34 );
35
36 CREATE TABLE Bewertung (
37     BewertungsID NUMBER(9),
38     BRezeptID NUMBER(9) NOT NULL,
39     Einfachheit NUMBER(1) NOT NULL, -- 1-5, zwingend, wenn man eine Bewertung erstellt
40     Geschmack NUMBER(1) NOT NULL, -- analog zur Einfachheit
41     Datum DATE NOT NULL,
42     Kommentar VARCHAR(1000),
43     BewerterID NUMBER(9) NOT NULL,
44     CONSTRAINT Bewertung_PK PRIMARY KEY (BewertungsID),
45     CONSTRAINT range_check_Einfachheit CHECK (Einfachheit >= 1 AND Einfachheit <= 5),
46     CONSTRAINT range_check_Geschmack CHECK (Geschmack >= 1 AND Geschmack <= 5)
47 );
48
49 CREATE TABLE Einheit (
50     EinheitID NUMBER(9),
51     Bezeichnung VARCHAR(30) NOT NULL,
52     CONSTRAINT Einheit_PK PRIMARY KEY (EinheitID)
53 );
54
55 CREATE TABLE Bild (
56     BildID NUMBER(9),
57     Dateiname VARCHAR(50) NOT NULL,
58     Verzeichnis VARCHAR(100) NOT NULL,
59     CONSTRAINT Bild_PK PRIMARY KEY (BildID)
60 );
61
62 CREATE TABLE Kochanweisung (

```

```

63     AnweisungsID NUMBER(9),
64     Zeit NUMBER(5) NOT NULL, -- kann in min gespeichert werden
65     Text VARCHAR(1000),
66     CONSTRAINT Kochanweisung_PK PRIMARY KEY (AnweisungsID)
67 );
68
69
70 -- Tabellen für die m:n Beziehungen
71
72
73 CREATE TABLE vergibt (
74     RezeptID NUMBER(9),
75     PersonID NUMBER(9),
76     LabelID NUMBER(9),
77     CONSTRAINT vergibt_PK PRIMARY KEY (RezeptID, PersonID, LabelID)
78 );
79
80 CREATE TABLE verwendet (
81     ZutatID NUMBER(9),
82     RezeptID NUMBER(9),
83     Menge NUMBER(4) NOT NULL,
84     CONSTRAINT verwendet_PK PRIMARY KEY (ZutatID, RezeptID)
85 );
86
87
88 CREATE TABLE entspricht (
89     EinheitID1 NUMBER(9),
90     EinheitID2 NUMBER(9),
91     Faktor NUMBER(12,6) NOT NULL, -- genug Nachkommastellen für präzise Umrechnung
92     CONSTRAINT entspricht_PK PRIMARY KEY (EinheitID1, EinheitID2)
93 );
94
95 CREATE TABLE Rezept_zeigt (
96     RezeptID NUMBER(9),
97     BildID NUMBER(9),
98     Reihenfolge NUMBER(4) NOT NULL,
99     CONSTRAINT Rezept_zeigt_PK PRIMARY KEY (RezeptID, BildID)
100 );
101
102
103 CREATE TABLE besteht_aus (
104     RezeptID NUMBER(9),
105     AnweisungsID NUMBER(9),
106     Reihenfolge NUMBER(4) NOT NULL,
107     CONSTRAINT bestehtAus_PK PRIMARY KEY (RezeptID, AnweisungsID)
108 );
109
110
111
112 CREATE TABLE Anweisung_zeigt (
113     AnweisungsID NUMBER(9),
114     BildID NUMBER(9),
115     Reihenfolge NUMBER(4) NOT NULL,
116     CONSTRAINT Anweisung_zeigt_PK PRIMARY KEY (AnweisungsID, BildID)
117 );
118 );

```

2.1 Ausgabe / Ausführung auf der Datenbank

Aus Gründen der Übersichtlichkeit kopiert und nicht gescreenshottet.

```

1  Table ZUTAT created.
2  Table REZEPT created.
3  Table MATERIALZUTAT created.
4  Table PERSON created.
5  Table LABEL created.

```



```
6  Table BEWERTUNG created.
7  Table EINHEIT created.
8  Table BILD created.
9  Table KOCHANWEISUNG created.
10 Table VERGIBT created.
11 Table VERWENDET created.
12 Table ENTSPRICHT created.
13 Table REZEPT_ZEIGT created.
14 Table BESTEHT_AUS created.
15 Table ANWEISUNG_ZEIGT created.
```

3. Rezeptdatenbank – Schemaevolution

```

1  ALTER TABLE Zutat ADD (
2      CONSTRAINT Zutat_ZEinheitID_FK FOREIGN KEY (ZEinheitID)
3          REFERENCES Einheit(EinheitID)
4  );
5
6  ALTER TABLE Rezept ADD (
7      CONSTRAINT Rezept_VerfasserID_FK FOREIGN KEY (VerfasserID)
8          REFERENCES Person(PersonID),
9      CONSTRAINT Rezept_RezeptID_FK FOREIGN KEY (RezeptID)
10         REFERENCES Zutat(ZutatID)
11  );
12
13 ALTER TABLE Materialzutat ADD (
14     CONSTRAINT Materialzutat_MaterialID_FK FOREIGN KEY (MaterialID)
15         REFERENCES Zutat(ZutatID)
16 );
17
18 -- Person passt bereits
19 -- Label passt bereits
20
21 ALTER TABLE Bewertung ADD (
22     CONSTRAINT Bewertung_BRezeptID_FK FOREIGN KEY (BRezeptID)
23         REFERENCES Rezept(RezeptID),
24     CONSTRAINT Bewertung_BewerterID_FK FOREIGN KEY (BewerterID)
25         REFERENCES Person(PersonID)
26 );
27
28 -- Einheit passt
29 -- Bild passt
30 -- Kochanweisung passt
31
32 ----- N:M Relationen updaten
33
34 ALTER TABLE vergibt ADD (
35     CONSTRAINT vergibt_RezeptID_FK FOREIGN KEY (RezeptID)
36         REFERENCES Rezept(RezeptID),
37     CONSTRAINT vergibt_PersonID_FK FOREIGN KEY (PersonID)
38         REFERENCES Person(PersonID),
39     CONSTRAINT vergibt_LabelID_FK FOREIGN KEY (LabelID)
40         REFERENCES Label(LabelID)
41 );
42
43 ALTER TABLE verwendet ADD (
44     CONSTRAINT verwendet_ZutatID_FK FOREIGN KEY (ZutatID)
45         REFERENCES Zutat(ZutatID),
46     CONSTRAINT verwendet_RezeptID_FK FOREIGN KEY (RezeptID)
47         REFERENCES Rezept(RezeptID)
48 );
49
50 ALTER TABLE entspricht ADD (
51     CONSTRAINT entspricht_EinheitID1_FK FOREIGN KEY (EinheitID1)
52         REFERENCES Einheit(EinheitID),
53     CONSTRAINT entspricht_EinheitID2_FK FOREIGN KEY (EinheitID2)
54         REFERENCES Einheit(EinheitID)
55 );
56
57 ALTER TABLE Rezept_zeigt ADD (
58     CONSTRAINT Rezept_zeigt_RezeptID_FK FOREIGN KEY (RezeptID)
59         REFERENCES Rezept(RezeptID),
60     CONSTRAINT Rezept_zeigt_BildID_FK FOREIGN KEY (BildID)
61         REFERENCES Bild(BildID)
62 );
63

```

```

64 ALTER TABLE besteht_aus ADD (
65     CONSTRAINT besteht_aus_RezeptID FOREIGN KEY (RezeptID)
66     REFERENCES Rezept(RezeptID),
67     CONSTRAINT besteht_aus_AnweisungsID FOREIGN KEY (AnweisungsID)
68     REFERENCES Kochanweisung(AnweisungsID)
69 );
70
71
72 ALTER TABLE anweisung_zeigt ADD (
73     CONSTRAINT anweisung_zeigt_BildID FOREIGN KEY (BildID)
74     REFERENCES Bild(BildID),
75     CONSTRAINT anweisung_zeigt_AnweisungsID FOREIGN KEY (AnweisungsID)
76     REFERENCES Kochanweisung(AnweisungsID)
77 );
78
79 ALTER TABLE Bewertung DROP COLUMN Geschmack;
80
81 ALTER TABLE Rezept ADD CONSTRAINT Rezept_Anzahl_range_check CHECK (Anzahl >= 1 AND Anzahl <= 12);

```

3.1 Ausgabe / Ausführung auf der Datenbank

```

1  Table ZUTAT altered.
2  Table REZEPT altered.
3  Table MATERIALZUTAT altered.
4  Table BEWERTUNG altered.
5  Table VERGIBT altered.
6  Table VERWENDET altered.
7  Table ENTSPRICHT altered.
8  Table REZEPT_ZEIGT altered.
9  Table BESTEHT_AUS altered.
10 Table ANWEISUNG_ZEIGT altered.
11 Table BEWERTUNG altered.
12 Table REZEPT altered.

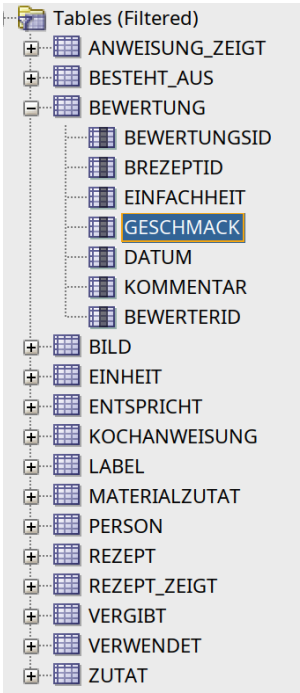
```

Tests

vor dem Aufruf von 3):

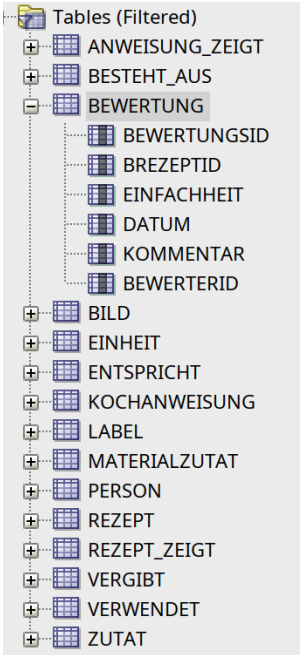
- Tabellen existieren
- Keine Fremdschlüssel
- Einschränkung der Personenanzahl ist nicht vorhanden

	CONSTRAINT_NAME	CONSTRAINT_TYPE	SEARCH_CONDITION
1	REZEPT_PK	Primary_Key	(null)
2	SYS_C001210361	Check	"BESCHREIBUNG" IS NOT NULL
3	SYS_C001210362	Check	"ANZAHL" IS NOT NULL
4	SYS_C001210363	Check	"VERFASSEID" IS NOT NULL



nach dem Aufruf:

- Geschmack ist weg
- Anzahl ist zwischen 1 und 12
- Fremdschlüssel sind da



CONSTRAINT_NAME	CONSTRAINT_TYPE	SEARCH_CONDITION
1 REZEPT_ANZAHL_RANGE_CHECK	Check	Anzahl >= 1 AND Anzahl <= 12
2 REZEPT_PK	Primary_Key	(null)
3 REZEPT_REZEPTID_FK	Foreign_Key	(null)
4 REZEPT_VERFASSERID_FK	Foreign_Key	(null)
5 SYS_C001210361	Check	"BESCHREIBUNG" IS NOT NULL
6 SYS_C001210362	Check	"ANZAHL" IS NOT NULL
7 SYS_C001210363	Check	"VERFASSERID" IS NOT NULL

4. Rezeptdatenbank – Löschen von Tabellen

```

1  DROP TABLE Anweisung_zeigt;
2  DROP TABLE besteht_aus;
3  DROP TABLE rezept_zeigt;
4  DROP TABLE entspricht;
5  DROP TABLE verwendet;
6  DROP TABLE vergibt;
7
8  DROP TABLE Bewertung;
9  DROP TABLE Rezept;
10 DROP TABLE Materialzutat;
11 DROP TABLE Person;
12 DROP TABLE Zutat;
13 DROP TABLE Label;
14 DROP TABLE Einheit;
15 DROP TABLE Bild;
16 DROP TABLE Kochanweisung;

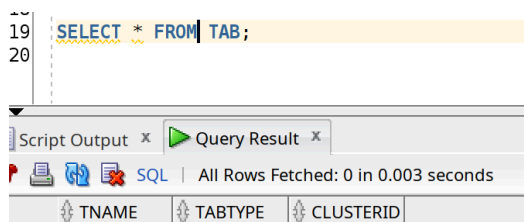
```

4.1 Ausgabe / Ausführung auf der Datenbank

```

1  Table ANWEISUNG_ZEIGT dropped.
2  Table BESTEHT_AUS dropped.
3  Table REZEPT_ZEIGT dropped.
4  Table ENTSPRICHT dropped.
5  Table VERWENDET dropped.
6  Table VERGIBT dropped.
7  Table BEWERTUNG dropped.
8  Table REZEPT dropped.
9  Table MATERIALZUTAT dropped.
10 Table PERSON dropped.
11 Table ZUTAT dropped.
12 Table LABEL dropped.
13 Table EINHEIT dropped.
14 Table BILD dropped.
15 Table KOCHANWEISUNG dropped.

```



--> kein Output, weil gelöscht

Aufgabe 5 Relationales Modell

Messungen(LogID:Number, GeräteID:Number, Geräte_Bezeichnung:VARCHAR, Einbau_Datum:Date, Messwert:Number(5,2), Einheit:VARCHAR, Messzeit:DATE, Status:Number(1), Aktiv:Number(1), Laufende_Kosten:Number(6,2), Unterer_Grenzwert:Number(5,2), Oberer_Grenzwert:Number(5,2))

1. NF und 2. NF erfüllt.

funktionale Abhängigkeiten:

{GeräteID} -> {Geräte_Bezeichnung, Einbau_Datum, Unterer_Grenzwert, Oberer_Grenzwert, Einheit, Laufende_Kosten}

{LogID} -> {GeräteID, Messwert, Messzeit, Status, Aktiv}

Es gibt also noch transitive Abhängigkeiten.

3. NF:

Messungen(LogID:Number, GeräteID:Number -> GeräteID.Gerät (ON DELETE CASCADE), Messwert:Number, Messzeit:DATE, StatusID:Number(1) -> StatusID.Status, Aktiv:Number(1))
 -- Aktiv speichert 0 für false und 1 für true

Gerät(GeräteID:Number, Geräte_Bezeichnung:VARCHAR, Einbau_Datum:Date, Unterer_Grenzwert:Number, Oberer_Grenzwert:Number, EinheitID:Number -> EinheitID.Einheit, Laufende_Kosten:Number)

Einheit(EinheitID:Number, Einheit:VARCHAR)

5. Erstellung DDL für gegebene Daten

```

1  CREATE TABLE Einheit (
2      EinheitID NUMBER(9),
3      Einheit VARCHAR(20),
4      CONSTRAINT Einheit_PK PRIMARY KEY (EinheitID)
5  );
6
7
8  CREATE TABLE Gerät (
9      GeräteID NUMBER(9),
10     Geräte_Bezeichnung VARCHAR(30) NOT NULL,
11     Einbau_Datum DATE, -- kann null sein, vll wird das Gerät vor dem Einbau im System registriert
12     Unterer_Grenzwert NUMBER(5,2), -- vll gibts noch keine Grenzwert, deshalb NULL erlaubt
13     Oberer_Grenzwert NUMBER(5,2),
14     Laufende_Kosten NUMBER(6,2), -- vll sind Kosten noch nicht verfügbar
15     EinheitID NUMBER(9) NOT NULL, -- FK
16     CONSTRAINT Gerät_PK PRIMARY KEY (GeräteID),
17     CONSTRAINT Laufende_Kosten_check CHECK (Laufende_Kosten >= 0),
18     CONSTRAINT Gerät_EinheitID_FK FOREIGN KEY (EinheitID)
19         REFERENCES Einheit(EinheitID)
20 );
21
22 CREATE TABLE Messungen (
23     LogID NUMBER(9),
24     GeräteID NUMBER(9), -- FK
25     Messwert NUMBER(5,2),
26     Messzeit DATE NOT NULL, -- date speichert auch Uhrzeit
27     StatusID NUMBER(1) NOT NULL, -- es gibt nur 3 Status
28     Aktiv NUMBER(1) DEFAULT 1 NOT NULL, -- false := 0, true := 1 | hier keine eigene Tabelle,
    effizienter
29     CONSTRAINT Messungen_PK PRIMARY KEY (LogID),
30     CONSTRAINT Messungen_GeräteID_FK FOREIGN KEY (GeräteID)
31         REFERENCES Gerät(GeräteID)
32         ON DELETE CASCADE,
33     CONSTRAINT StatusID_check CHECK (StatusID >= 0 AND StatusID <= 3),
34     CONSTRAINT Aktiv_check CHECK (Aktiv >= 0 AND Aktiv <= 1),
35     CONSTRAINT Messwert_check CHECK (Messwert IS NOT NULL OR StatusID = 2)
36 )

```

5.1 Ausgabe / Ausführung auf der Datenbank

```

1  Table EINHEIT created.
2  Table GERÄT created.
3  Table MESSUNGEN created.

```